Cosmos vial inspection

Version 1 pilot and Version 2 preparation

Technical report | Prepared for the VyFlare team | 29 October 2026

Version 1 is a trained color-sequence adapter. Version 2 is a completed field-data audit and review pipeline, not a second trained model. The latest work improves traceability and exposes label problems. It does not establish better prediction accuracy or add ingredient, volume, or recipe recognition to the deployed model.

Dimension	Version 1: trained pilot	Version 2: field preparation
Primary task	Describe visible liquid color sequences.	Prepare evidence for future ingredient, timing, volume, and recipe analysis.
Supervision	30 human-reviewed organizer clips; 21 train, nine validation.	30 field-vial records; all observations remain unverified at the user's request.
Model changes	Rank-8 LoRA trained on the extracted Cosmos3-Nano reasoner.	No weight updates; existing adapter and tester preserved.
Evidence of success	Measured improvement on nine same-session development clips.	Reproducible audit, source hashes, review flags, and validation checks.
Main limitation	Narrow appearance target and no independent-session accuracy.	Zero eligible training/test records until observations and links are confirmed.

Why the approach changed

The project expanded from describing appearance to identifying incoming ingredients, measuring dispense duration and amount, retaining prior-ingredient history, and matching known recipes through a backend. Color-only supervision cannot establish those capabilities. The second stage therefore separates intended recipe values from provisional observations and measured delivery. [E1-E4]

Current operational status

The first adapter remains available through the existing private-network tester. The field workflow produced private review records; it did not cut training clips from unverified links, launch training, replace the tester, or implement getRecipe. [E2-E4]

1. Version 1: what was built and measured

We extracted the understanding pathway from nvidia/Cosmos3-Nano and mapped and shape-checked all 750 expected reasoner tensors, including 351 vision tensors. The source checkpoint was preserved. No substitute foundation-model weights were used. [E1, E5]

Training component	Implemented setting
Adapter	7,667,712 trainable parameters; rank 8, alpha 16, dropout 0.05.
Trainable modules	Language-attention q/k/v/o projections only; base and vision weights frozen.
Optimization	BF16 base; SDPA; gradient checkpointing; AdamW LR 5e-5; accumulation four; 20 updates; seed 42.
Input and target	32 uniform frames per clip; 2,621,440 total video pixels; ordered observed_color_sequence JSON.
Partition	21 training clips; nine validation clips from recipes 6, 9, and 13. Both partitions share organizer video 5.

Nine-clip development metric	Base reasoner	Version 1 adapter
Exact color sequence	0/9 (0.0%)	4/9 (44.4%)
Exact final color label	3/9 (33.3%)	5/9 (55.6%)
Valid color JSON	7/9 (77.8%)	9/9 (100.0%)

These are same-session development agreement scores. They do not measure field accuracy, chemical identity, volume, recipe identification, or unknown-recipe detection. The nine examples are too few for a stable general accuracy estimate. [E1]

Where Version 1 performs usefully

It provides a working, reversible adaptation and a repeatable base-versus-adapter comparison. Some green and orange examples match human descriptions, and structured output became consistent across the nine evaluation clips. The browser tester supports experimentation and explicit reviewer feedback. Saved feedback is not automatically accepted as training truth.

Observed weaknesses

All three blue-to-dark-green targets became green. The orange-to-brown target for vial 26 became orange; vial 5 gained an extra yellow stage. Uniform sampling and coarse whole-clip labels limit temporal supervision. Photo mode was not trained or validated and previously returned a schema-invalid scalar color. [E1, E2]

2. Version 2: completed changes and audit

The corrected recipes.xlsx provides 52 recipe definitions, including intended amounts and dispense order. Its field sheet describes five vials each for recipes 41, 50, and 60, and 15 water-flush vials for recipe 99. field-data.zip contains six videos, six recording JSON files, and 30 photos. [E3, E4]

Recipe	T100 clear/water	T200 red	T400 blue	Expected order
41	50 mL	20 mL	-	T200 > T100
50	-	30 mL	40 mL	T200 > T400
60	30 mL	10 mL	30 mL	T200 > T400 > T100
99	50 mL	-	-	T100

The team-confirmed T200 reference is red. An orange-looking recorded mixture remains an observation to investigate; it is not silently changed to red. Recipe-bank amounts and recording metadata agree for these definitions, but agreement does not prove actual delivered quantities.

Implemented preparation

Added scripts/prepare_field_review.py and tests/test_field_review.py. The read-only importer writes a private recipe bank, recording inventory, per-vial review manifest, and source-hashed audit under data/local/field-review-v1/. It refuses to overwrite an existing output version. Original source files are unchanged. [E3]

Each record keeps intended amounts separate from provisional durations, observed order, and review notes. Missing values remain missing. All 30 records have reviewed=false, training_eligible=false, an empty answer, and an unassigned split. These are review records, not trainer-ready examples.

Audit finding	Extent	Interpretation
Expected/observed order conflict	15 vials	Vials 1-5, 11-15, 21-25 require source/order review.
Pour duration exceeds vial window	12 vials	Vials 7-9, 17-20, 26-30 have inconsistent timing definitions or values.
Required duration missing	Vial 1	T100 duration is unknown despite an intended T100 amount.
Video-link hypothesis incomplete	Vial 21	29 candidate links fit; one lacks a unique window. None is confirmed.

Candidate links assume a concatenated timeline using recording durations in chronological order. Boundary precision and timeline origins remain uncertain. No clips were cut using these links. Audit flags are not proof of a dispensing fault or contamination. [E3, E4]

3. Strengths, pain points, and failure modes

Issue	Version 1	Version 2: present state and remaining need
Appearance versus ingredient	Learns vial appearance; could mistake residual color for incoming ingredient if used beyond its target.	Separates recipe intent and visual observations. Ingredient/source assignments remain provisional; controller mapping or review is still needed.
Temporal detail	Whole-clip target lacks pour boundaries and may miss brief stages or shades.	Stores per-ingredient duration and vial windows, exposing inconsistencies. Correct start/end labels and a new event target contract are still needed.
Amount estimation	No volume target or flow calibration; cannot infer reliable mL from color.	Retains intended mL separately. Actual delivery and per-fluid calibration are absent; no volume capability has been added.
Recipe and history	No implemented recipe lookup, persistent ingredient history, or unknown-recipe decision.	Preserves recipe/flush context for review. Persistent state, backend matching, and getRecipe remain future implementation.
Evaluation leakage	Recipe groups separated, but one shared session may make validation easier than new field operation.	Keeps connected field batch in one session group with no fabricated 24/6 split. Independent confirmed sessions are still missing.
Operational usefulness	Working adapter, repeatable comparison, browser access; narrow and unvalidated outside pilot.	Better traceability and explicit review priorities; cannot offer improved predictions because it has no newly trained adapter.

What the first approach would miss for the expanded task

A vial can appear blue during a clear-water flush because colored liquid remains. Version 1 was trained to describe that appearance; using its color answer as ingredient truth could yield a wrong recipe match. The second workflow makes this mismatch visible, but has not solved ingredient perception. Likewise, two recipes may have similar final colors yet different amounts or orders. Only verified event evidence and measurements can distinguish them reliably.

Costs of the second approach

It needs more alignment work, selective human or controller verification, flow calibration, and independently recorded tests. Proposed backend and event-training components increase integration effort. These costs buy defensible labels and measurements; they do not guarantee a particular accuracy.

4. DGX Spark use, verification, and next experiment

Resource or check	Version 1	Version 2
Main training job	20 updates; 487.17 s (8.12 min) including load and evaluations.	No new training job or optimizer updates.
Optimizer-loop time	268.67 s (4.48 min), including batch processing.	Zero additional training time.
Smoke run	136.17 s (2.27 min), including load and evaluations; one update.	No additional GPU smoke training.
Combined recorded job wall time	623.34 s (10 min 23 s); not exclusive GPU-hours.	Read-only SSH resource check only.
Peak allocated GPU memory	19.54 GiB in main pilot; 19.44 GiB in smoke run.	No new training allocation measured.
Spark readiness	GB10; existing Linux/PyTorch/PEFT runtime used.	SSH reachable; base checkpoint present; 121 GiB host RAM, 69 GiB available at check.

Version 1 resource totals exclude checkpoint extraction, preparation, transfers, troubleshooting, and ongoing serving. Version 2 adds zero training steps and zero training GPU time; this excludes existing services and connection/check overhead. Whole-host utilization cannot be attributed to our work. Energy, monetary cost, and cumulative serving GPU-hours were not measured. [E1, E3, E5]

Troubleshooting and validation

Version 1 required strict checkpoint mapping, correct total-video pixel budgeting, and an asserted assistant-only loss boundary. PyTorch emitted a GB10 compatibility warning, but inference and backward updates completed. The latest local test run initially failed because sandbox temporary files were inaccessible; rerunning outside that sandbox passed: 22 discovered, 16 passed, six server tests skipped because Flask is absent locally. Six server tests had passed on Spark in the earlier tester work; the latest change did not rerun them remotely. [E1-E3]

Next experiment and release criteria

First confirm video links, ingredient/source identity, and pour boundaries. Obtain flow calibration or measured delivery for volume labels. Reserve independent sessions for evaluation; use development data for prompts, training length, and thresholds. Keep known-recipe generalization separate from unknown-recipe tests, which must hold recipes out of training and the known catalog.

Then implement validated event targets, train a separate LoRA adapter, and compare the unchanged base, Version 1, and the candidate. Measure ingredient accuracy, timing error, measured-volume error, recipe matching, and rejection of unknown or insufficient-evidence cases. Recipe retrieval and arithmetic belong in the backend. A match score should not be called a calibrated probability without held-out calibration.

5. Reproducibility and evidence

All version labels in this report describe project stages. There is no Version 2 model checkpoint or measured Version 2 improvement. The existing color-sequence trainer is not yet an ingredient-event trainer. The new review importer is deliberately separate so an unresolved field observation cannot silently become a target.

ID	Source and scope
E1	reports/spark-pilot-v1.md; outputs/spark-pilot-v1/run.json, smoke-run.json, base.json, adapted.json, and training.jsonl. Actual training configuration, resource measurements, and predictions.
E2	reports/model-tester-guide.md and existing tester implementation. Browser testing, optional feedback, and photo-mode limitations.
E3	reports/field-review-v1.md; scripts/prepare_field_review.py; tests/test_field_review.py; private data/local/field-review-v1/audit.json, review.jsonl, recipe-bank.json, and recordings.json. Latest preparation and audit.
E4	User-supplied C:/Users/vuelu/Downloads/recipes.xlsx, sheets Recipe_Bank, Verified_Recipes, and Recipes_41_50_60_99; C:/Users/vuelu/Downloads/field-data.zip recording JSON, media inventory, and sampled visual review. Provisional notes are evidence to review, not instructions or confirmed targets.
E5	Prior Cosmos technical report and artifact verification records under outputs/spark-pilot-v1/. Source checkpoint and adapter identity, tensor verification, and recorded Spark use.

Identities

Base model: nvidia/Cosmos3-Nano
Source snapshot: e59a53c25979a090fa8706c9acc0c254a6e89b92

Version 1 adapter SHA-256:
9dae56b981de7e7de200e39580e57cbcba81d3efdcf940f2e025e80a41a3a5f4

Corrected workbook SHA-256:
e95239f57cbe69835c136be1f98f4086b0cf3844654d12a4b58161c43b173b57

Field archive SHA-256:
b2bab5002945f59369b3f360f4f0a81dfbee16031232f9b21cfa157ffa58e79e

Prepared artifacts and branch

Branch: codex/cosmos3-nano-finetune. New preparation code, tests, and report are in the working tree; no new commit or deployment is asserted. Private datasets and model weights remain excluded from Git. Existing artifacts and adapters are preserved.

Decision supported by the evidence

Continue with targeted label verification and independent data collection before the next field-supervised weight update. Version 1 demonstrates that small-adapter training is practical on the Spark. Version 2 establishes a more defensible data foundation for the broader task, while explicitly showing which evidence is still missing.